



AGENTE AUTÔNOMO

Vivendo em Ubatuba

Documentação Técnica Completa — v5.1



Instagram YouTube Tavily Claude AI GitHub FTP

© 2026 Vivendo em Ubatuba

1. Visão Geral do Projeto

O Agente Autônomo **Vivendo em Ubatuba** é um sistema de inteligência artificial que automatiza completamente a gestão de conteúdo para redes sociais do perfil @vivendoemubatuba_oficial. O agente opera de forma independente, sem necessidade de intervenção humana diária, publicando conteúdo relevante, útil e atualizado sobre a cidade de Ubatuba — SP.

1.1 Propósito

O perfil 'Vivendo em Ubatuba' tem como objetivo ser a principal fonte de informação local para moradores e turistas da cidade, cobrindo temas como clima, praias, eventos culturais, gastronomia, natureza, turismo, surf e tradição caiçara. Como o proprietário do perfil não dispõe de tempo para criar conteúdo manualmente todos os dias, o agente foi desenvolvido para realizar esse trabalho de forma autônoma e inteligente.

1.2 Público-alvo do perfil

- Moradores de Ubatuba que buscam informações úteis do dia a dia
- Turistas planejando ou que estão visitando a cidade
- Amantes do litoral norte paulista e da cultura caiçara
- Surfistas e praticantes de esportes aquáticos

1.3 Calendário Editorial Automatizado

O agente segue uma estratégia editorial estruturada por dia da semana. Além das publicações no Instagram, cada post gera automaticamente um Short para o YouTube, totalizando até 3 publicações por dia nas duas plataformas.

Dia	Publicação 1	Publicação 2
Segunda a Quinta	■■ Story Clima/Tempo	■ ■ Story + Feed — Notícia destaque
Sexta-feira	■■ Story Clima/Tempo	■ ■ Story + Feed — Resumo da Semana
Sábado e Domingo	■■ Story Clima/Tempo	■ ■ Story + Feed — Notícia destaque

2. Arquitetura do Sistema

O pipeline do agente é composto por 6 etapas sequenciais que transformam dados brutos da internet em publicações formatadas nas redes sociais. **Na v5.1, o fluxo de vídeo foi atualizado:** o Story de Clima agora é publicado como vídeo MP4 via FTP, e a publicação no Instagram aguarda confirmação de status **FINISHED** antes de prosseguir.

Etapa	Descrição	Novidade v5.1
1 — Busca (Tavily)	2 camadas: fontes locais + busca geral por categoria	—
2 — IA (Claude)	3 prompts: Clima / Temático / Resumo da Semana	—
3 — Imagem (Pillow)	Card JPEG 1080x1080 (Feed) e 1080x1920 (Stories)	—
3b — Vídeo (MoviePy)	Converte card em MP4 10s com música de fundo	■ Usado no Story Clima
4a — Upload FTP	Sobe MP4 no servidor e obtém URL pública	■ Novo no Story Clima
4b — Upload ImgBB	Sobe JPEG para URL pública (feed e story temático)	—
5a — Instagram	Story Clima (vídeo) + Story/Feed Temático (imagem)	■ Polling FINISHED
5b — YouTube	Short MP4 10s publicado via upload resumável	■ Reutiliza vídeo gerado
6 — Histórico (.jsonl)	Registro de IDs e resultado de cada execução	—

2.1 Detalhe do Fluxo — Story Clima (v5.1)

O Story de Clima foi migrado de imagem estática para **vídeo MP4**, aumentando o alcance orgânico no Instagram. O novo fluxo interno é:

- Gera card JPEG 1080x1920 com **gerar_card()**
- Converte para MP4 10s com música via **gerar_video_short()**
- Faz upload do MP4 no servidor FTP via **fazer_upload_ftp()** → obtém URL pública
- Cria container de vídeo na API do Instagram via **_criar_container_video()**
- Aguarda status **FINISHED** via **_aguardar_processamento_video()** (polling a cada 10s, até 15 tentativas)
- Publica o container via **_publicar_container()**
- Deleta o MP4 do FTP via **deletar_ftp()**
- Reutiliza o mesmo **video_bytes_c** para publicar no YouTube (sem regerar o vídeo)

2.2 Módulos do Script Principal

Função	Módulo	Responsabilidade
buscar_clima_apenas()	Busca	Busca focada só em clima/tempo (2 queries)
buscar_noticias_ubatuba()	Busca	Busca geral em 2 camadas (11 queries)

buscar_noticias_semana()	Busca	Busca semanal para o Resumo de Sexta
_buscar_tavily()	Busca	Executa query na Tavily API com filtros
gerar_conteudo_clima()	IA	Gera conteúdo de clima via Claude
gerar_conteudo_tematico()	IA	Seleciona melhor tema e gera conteúdo
gerar_resumo_semana()	IA	Gera resumo com 5-10 notícias da semana
_chamar_claude()	IA	Interface comum com a API da Anthropic
gerar_card()	Imagem	Gera card com foto de fundo + texto + emoji
gerar_card_resumo()	Imagem	Card especial com lista de notícias da semana
gerar_video_short()	Vídeo	Converte imagem em MP4 10s com música
fazer_upload_ftp()	FTP	Upload do MP4 no servidor FTP (Story Clima) ■ v5.1
deletar_ftp()	FTP	Apaga MP4 do FTP após publicação ■ v5.1
_aguardar_processamento_video()	Instagram	Polling status FINISHED antes de publicar ■ v5.1
fazer_upload_imgbb()	Upload	Sobe imagem para ImgBB e retorna URL
_criar_container()	Instagram	Cria container de imagem na API Meta
_criar_container_video()	Instagram	Cria container de vídeo na API Meta ■ v5.1
_publicar_container()	Instagram	Publica o container criado
publicar_youtube_short()	YouTube	Upload e publicação do Short no YouTube
executar_agente()	Orquestrador	Coordena todo o pipeline diário

3. Plataformas e APIs Envolvidas

3.1 Tavily API — Busca de Notícias

A Tavily é uma API de busca web otimizada para inteligência artificial. Retorna resultados já resumidos e limpos, ideais para alimentar um modelo de linguagem.

Atributo	Detalhe
Site	tavily.com
Plano utilizado	Free — 1.000 requisições/mês (suficiente para uso diário)
Custo estimado	R\$ 0 (plano gratuito)
Queries por execução	2 (clima) ou 11 (notícias gerais) ou 6 (resumo semana)
Parâmetro chave	include_domains: prioriza ubatubatimes.com.br e ubatuba.sp.gov.br
Variável de ambiente	TAVILY_API_KEY

3.2 Anthropic API (Claude) — Inteligência Artificial

Atributo	Detalhe
Modelo	claude-opus-4-5 (mais capaz para curadoria editorial)
Site	console.anthropic.com
Custo estimado	~R\$ 1,50/mês (3 chamadas/dia x ~30 dias)
Prompts ativos	3: PROMPT_TEMPO / PROMPT_TEMATICO / PROMPT_RESUMO_SEMANA
Formato de saída	JSON estruturado: titulo, subtítulo, corpo, cta, hashtags, tema, confiança
Variável de ambiente	ANTHROPIC_API_KEY

3.3 Meta Graph API (Instagram)

Atributo	Detalhe
Versão da API	Graph API v21.0
Conta requerida	Instagram Business ou Creator (não funciona com conta pessoal)
Publicações/dia	Limite de 25 posts por dia via API
Processo publicação imagem	2 etapas: criar container (/media) → publicar (/media_publish)
Processo publicação vídeo	Criar container → aguardar FINISHED → publicar ■ v5.1
Token	Access Token de longa duração (~60 dias) — requer renovação periódica
Variáveis de ambiente	IG_ACCESS_TOKEN, IG_USER_ID

3.4 YouTube Data API v3 (YouTube Shorts)

Atributo	Detalhe
API	YouTube Data API v3 — upload resumável
Autenticação	OAuth2 com Refresh Token permanente (app em produção)
Formato do vídeo	MP4 vertical 1080x1920px, 10 segundos, 24fps, codec H264+AAC
Identificação Short	Automática: vídeo vertical ≤60s com #Shorts na descrição
Título padrão	UBATUBA - [título gerado pela IA]
Música de fundo	Arquivo MP3 royalty-free em music/background_music.mp3
Variáveis de ambiente	YOUTUBE_CLIENT_ID, YOUTUBE_CLIENT_SECRET, YOUTUBE_REFRESH_TOKEN

3.5 Servidor FTP — Hospedagem Temporária de Vídeos ■ v5.1

O FTP é usado para hospedar temporariamente os vídeos MP4 do Story de Clima antes de enviá-los à API do Instagram, que exige uma URL pública acessível. Após a publicação, o arquivo é deletado automaticamente do servidor.

Atributo	Detalhe
Host padrão	ftp.cojanez.cnt.br
Pasta de destino	/Web/wilianjanez/movies/auto_post_vivendo
URL pública base	https://wilianjanez.inf.br/movies/auto_post_vivendo
Expiração	Arquivo deletado automaticamente após publicação (deletar_ftp())
Variáveis de ambiente	FTP_HOST, FTP_USER, FTP_PASS, FTP_PASTA, FTP_URL

3.6 ImgBB — Hospedagem de Imagens

Atributo	Detalhe
Site	imgbb.com — gratuito, sem cartão de crédito
Uso	Hospedar cards JPEG do Feed e Story Temático (não o Story Clima)
Expiração	86400 segundos (24 horas) — suficiente para o post ser criado
Upload	Via base64 na API REST — sem necessidade de armazenamento local
Variável de ambiente	IMGBB_API_KEY

3.7 GitHub Actions — Agendamento e Execução

Atributo	Detalhe
Horário de execução	06h00 BRT (09h00 UTC) — todos os dias

Servidor	ubuntu-latest — Linux gratuito fornecido pelo GitHub
Timeout	30 minutos — encerra automaticamente se travar
Repositório	Público (necessário para agendamento gratuito)
Credenciais	Armazenadas em GitHub Secrets — nunca expostas no código
Logs	Salvos como artefatos por 30 dias — acessíveis na aba Actions

4. Geração de Imagens e Vídeos (Cards Visuais)

4.1 Design dos Cards

Cada card segue um design editorial consistente, gerado dinamicamente com a biblioteca Pillow (Python), combinando uma foto de fundo de Ubatuba com texto, emojis coloridos e elementos gráficos.

Formato	Dimensão	Uso
Feed Instagram	1080 x 1080px (1:1)	Post no feed com legenda completa
Stories Instagram	1080 x 1920px (9:16)	Story Temático — imagem estática
Story Clima	1080 x 1920px → MP4 10s	Story Clima — vídeo com música ■ v5.1
YouTube Shorts	MP4 1080x1920px, 10s, 24fps	Vídeo curto com música de fundo
Card Resumo Sexta	Ambos os formatos	Layout especial com lista de notícias

4.2 Fontes Utilizadas

- **Poppins-Bold.ttf** — títulos grandes e marca (153 KB)
- **DejaVuSans.ttf** — subtítulos e corpo do texto (742 KB)
- **NotoColorEmoji.ttf** — emojis coloridos reais (11 MB) — renderizado em 109px e redimensionado

4.3 Acervo de Fotos de Fundo

- 17 fotos de Ubatuba no acervo atual
- Hospedadas no servidor próprio (wilianjanez.inf.br/images/images_autopost/)
- Sorteio via `random.choice()` com seed baseado no relógio do sistema
- Feed e Story do mesmo dia sempre usam a mesma foto (consistência visual)
- Para adicionar novas fotos: suba no servidor e adicione a URL na variável `FUNDO_URLS` do GitHub Secrets

5. Configuração e Variáveis de Ambiente

Todas as credenciais e configurações são armazenadas em variáveis de ambiente (.env local ou GitHub Secrets para produção). **NUNCA devem ser colocadas diretamente no código.**

Variável	Serviço	Como obter
TAVILY_API_KEY	Tavily	tavily.com → Sign up → API Keys
ANTHROPIC_API_KEY	Anthropic	console.anthropic.com → API Keys
IG_ACCESS_TOKEN	Instagram	developers.facebook.com → Graph API Explorer
IG_USER_ID	Instagram	ID numérico da conta Business (ex: 17841449...)
IMGBB_API_KEY	ImgBB	imgbb.com → About → API → Get API key
FUNDO_URLS	Próprio	URLs das fotos separadas por vírgula (sem quebra de linha)
YOUTUBE_CLIENT_ID	Google	console.cloud.google.com → Credentials → App Desktop
YOUTUBE_CLIENT_SECRET	Google	console.cloud.google.com → Credentials → App Desktop
YOUTUBE_REFRESH_TOKEN	Google	Executar gerar_token_youtube.py uma vez
FONT_BOLD_PATH	Sistema	Caminho para Poppins-Bold.ttf
FONT_REG_PATH	Sistema	Caminho para DejaVuSans.ttf
FONT_EMOJI_PATH	Sistema	Caminho para NotoColorEmoji.ttf
FTP_HOST ■ v5.1	FTP	Servidor FTP (ex: ftp.cojanez.cnt.br)
FTP_USER ■ v5.1	FTP	Usuário FTP (ex: cojanez)
FTP_PASS ■ v5.1	FTP	Senha FTP — OBRIGATÓRIA, não deixar vazia
FTP_PASTA ■ v5.1	FTP	Caminho remoto (ex: /Web/wilianjanez/movies/auto_post_vivendo)
FTP_URL ■ v5.1	FTP	URL base pública (ex: https://wilianjanez.inf.br/movies/auto_post_vivendo)

■■ ATENÇÃO — Token do Instagram expira a cada 60 dias! Configure um lembrete recorrente para renovar antes do vencimento. Como renovar: developers.facebook.com → seu app → Instagram Basic Display → Gerar token → atualizar Secret IG_ACCESS_TOKEN no GitHub.

6. Correções de Bugs — v5.1

Esta seção documenta os três bugs identificados e corrigidos durante a atualização para a v5.1, bem como suas causas raiz.

#	Erro	Causa raiz	Correção aplicada
1	NameError: name 'bytes_sc'	Variável não existente usada no bloco YouTube	Substituir o nome da variável por bytes_img gerado; se
2	YouTube Clima sempre None	YouTube não possui o atributo bytes_c	Reservados para a liberação de dados no plot, q
3	530 User cannot log in (FTP)	FTP_HOST, FTP_USER, FTP_PASS, FTP_PORT	Adição das 5 variáveis de configuração
4	400 Media ID is not available	Instagram não possui o atributo media_id	Adição da função de geração de ID para o Instagram (eco

7. Workflow GitHub Actions — agente_ubatuba.yml

O arquivo abaixo é a versão atualizada do workflow. A principal mudança em relação à versão anterior é a adição das 5 variáveis FTP no step de configuração de ambiente.

```
name: Agente Vivendo em Ubatuba
on:
  schedule:
    - cron: "0 9 * * *" # 06h00 BRT (09h00 UTC)
  workflow_dispatch:
jobs:
  publicar-post:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Instalar dependencias
        run: |
          pip install requests python-dotenv Pillow numpy
          pip install moviepy==1.0.3
          pip install google-auth-oauthlib google-api-python-client
      - name: Configurar variaveis de ambiente
        run: |
          echo "TAVILY_API_KEY=${{ secrets.TAVILY_API_KEY }}" >> .env
          echo "ANTHROPIC_API_KEY=${{ secrets.ANTHROPIC_API_KEY }}" >> .env
          echo "IG_ACCESS_TOKEN=${{ secrets.IG_ACCESS_TOKEN }}" >> .env
          echo "IG_USER_ID=${{ secrets.IG_USER_ID }}" >> .env
          echo "IMGBB_API_KEY=${{ secrets.IMGBB_API_KEY }}" >> .env
          echo "FUNDO_URLS=${{ secrets.FUNDO_URLS }}" >> .env
          echo "FONT_BOLD_PATH=${{ secrets.FONT_BOLD_PATH }}" >> .env
          echo "FONT_REG_PATH=${{ secrets.FONT_REG_PATH }}" >> .env
          echo "FONT_EMOJI_PATH=${{ secrets.FONT_EMOJI_PATH }}" >> .env
          echo "YOUTUBE_CLIENT_ID=${{ secrets.YOUTUBE_CLIENT_ID }}" >> .env
          echo "YOUTUBE_CLIENT_SECRET=${{ secrets.YOUTUBE_CLIENT_SECRET }}" >> .en
v
          echo "YOUTUBE_REFRESH_TOKEN=${{ secrets.YOUTUBE_REFRESH_TOKEN }}" >> .en
v
          # FTP -- adicionado na v5.1
          echo "FTP_HOST=${{ secrets.FTP_HOST }}" >> .env
          echo "FTP_USER=${{ secrets.FTP_USER }}" >> .env
          echo "FTP_PASS=${{ secrets.FTP_PASS }}" >> .env
          echo "FTP_PASTA=${{ secrets.FTP_PASTA }}" >> .env
          echo "FTP_URL=${{ secrets.FTP_URL }}" >> .env
      - name: Executar agente
        run: python agente_ubatuba.py
      - name: Salvar log
```

```
if: always()
uses: actions/upload-artifact@v4
with:
  name: log-${{ github.run_id }}
  path: agente_ubatuba.log
  retention-days: 30
```

8. Estrutura de Arquivos do Projeto

```
AutoPostagem_VivendoEmUbatuba/
|
+-- agente_ubatuba.py          <- Script principal (pipeline completo)
+-- gerar_token_youtube.py     <- Gera Refresh Token YouTube (rodar 1x)
+-- testar_token_youtube.py    <- Testa autenticacao YouTube
|
+-- .env                       <- Credenciais locais (NUNCA subir no Git)
+-- .gitignore                 <- Protege .env e logs
+-- README.md                  <- Documentacao basica
|
+-- fonts/
|   +-- Poppins-Bold.ttf       (153 KB)
|   +-- DejaVuSans.ttf         (742 KB)
|   +-- NotoColorEmoji.ttf     (11 MB)
|
+-- music/
|   +-- background_music.mp3   (royalty-free)
|
+-- fotos/
|   +-- foto01.png ... foto17.png
|
+-- .github/
|   +-- workflows/
|       +-- agente_ubatuba.yml <- Agendamento GitHub Actions (06h BRT)
```

9. Estimativa de Custos Mensais

Servico	Plano	Custo/mes	Observacao
Tavily API	Free	R\$ 0	1.000 req/mes — suficiente
Anthropic (Claude)	Pay-as-you-go	~R\$ 1,50	3 chamadas/dia x 30 dias
Meta Graph API	Gratuito	R\$ 0	Requer conta Business
YouTube Data API	Gratuito	R\$ 0	10.000 unidades/dia de cota
ImgBB	Gratuito	R\$ 0	Upload ilimitado
Servidor FTP	Hospedagem propria	R\$ 0	Ja utilizado para outros fins
GitHub Actions	Free (repo publico)	R\$ 0	2.000 min/mes inclusos
TOTAL ESTIMADO	—	~R\$ 1,50	Por mes

10. Guia de Manutenção

10.1 Tarefas Recorrentes

Frequencia	Tarefa	Como fazer
A cada 60 dias	Renovar token Instagram	developers.facebook.com → seu app → Gerar token → atualizar Secret IG_AC
Quando quiser	Adicionar novas fotos	Subir foto no servidor → adicionar URL ao Secret FUNDO_URLS
Se falhar	Verificar log	GitHub → Actions → ultima execucao → ver log detalhado
Anual	Revisar temas	Ajustar PROMPT_TEMATICO e lista de temas conforme sazonalidade

10.2 Como Monitorar as Execuções

- Acesse github.com/wilianjanez/AutoPostagem_VivendoEmUbatuba/actions
- Execuções automáticas aparecem com evento 'schedule'
- Execuções manuais aparecem com evento 'workflow_dispatch'
- Clique em qualquer execução para ver o log detalhado
- Logs são salvos como artefatos por 30 dias

10.3 Dependências Python

Biblioteca	Versao	Uso no projeto
requests	latest	Chamadas HTTP para todas as APIs
python-dotenv	latest	Leitura do arquivo .env
Pillow	latest	Geracao dos cards de imagem (JPEG)
numpy	latest	Conversao de imagem para array (moviepy)
moviepy	1.0.3	Conversao de imagem em video MP4 para YouTube e Story Clima
google-auth-oauthlib	latest	Geracao do Refresh Token YouTube
google-api-python-client	latest	Cliente oficial da API do Google/YouTube

Instalação completa:

```
pip install requests python-dotenv Pillow numpy moviepy==1.0.3  
google-auth-oauthlib google-api-python-client google-auth-httpplib2
```